FRONT-END WEB DEVELOPMENT NC III

Offline Assessment Handbook + Online Palengke Build Guide

Practice edition based on the supplied TESDA SAG and Training Regulations

25 October 2026

Start here: your offline survival plan

Goal. Be able to create, explain, modify, test, and document a responsive front-end site using HTML, CSS, JavaScript, Vue, Git, and an API **without needing the public internet during practice**. This is an assessment preparation kit, not a prediction of the exact assessment task or a guarantee of competency certification.

What is in the companion ZIP? A complete Vite + Vue app; an optional local Node.js mock API; a deliberately separate localStorage-based demo mode; a no-install HTML/CSS/JavaScript fallback; reusable validation; a sample project plan; an API contract; and a manual test plan. All code is editable. The ZIP **does not include `node_modules`**; obtain packages before disconnecting.

Source alignment. The supplied TESDA *Self-Assessment Guide* (SAG), pp. 1-5, covers methods, version control, testing, security, documentation, responsive HTML/CSS, JavaScript/DOM/API, frameworks, and performance. The supplied *Training Regulations*, core units **ICT251307** and **ICT251308**, explain the competency evidence. The teaching examples below are my practical implementation, **not code mandated by TESDA**. Different assessors can provide different projects, mockups, APIs, tools, or workflows.

1. Install and save these items before your connection disappears

Priority	Install or save	Why it matters	Offline proof
Essential	**Node.js 24 LTS** (includes npm)	Runs Vite, the local API, and tests	`node -v` and `npm -v`
Essential	**Visual Studio Code**	Edits `.vue`, `.js`, `.html`, `.css`, `.md`	Open this project with Wi-Fi off
Essential	**Git for Windows**	Required practice for version control	`git --version` and `git status`
Essential	**Chrome / Edge or another current browser**	DevTools: Console, Network, Elements, device emulation	Open `http://127.0.0.1:5173`
Essential	**This kit + `node_modules` + `package-lock.json`**	Necessary for a truly offline Vue/Vite installation	`npm run dev`, `npm run build`
Recommended	**Vue - Official** VS Code extension (`Vue.volar`)	Vue syntax and template intelligence	Edit `App.vue` without connection
Recommended	**Prettier** (`esbenp.prettier-vscode`)	Consistent formatting	Format document while offline
Recommended	**ESLint** (`dbaeumer.vscode-eslint`)	Helps spot coding issues, when configured	Inspect diagnostics offline
Recommended	VSIX copies of important extensions	Reinstall extensions without the marketplace	`code --install-extension <file.vsix>`
Recommended	Copies of SAG, TR, project source, screenshots, and notes	Offline reference material and evidence	Open all files locally
Optional	Postman / Bruno or a REST client	Manual endpoint tests; not needed for mock API	Test `localhost:3000`

Before leaving: verify the relevant installers are downloaded for *your operating system*, not just bookmarked. The VS Code Extensions pane has **Download VSIX** and **Install from VSIX** actions for offline transfer. Browser tools and PowerShell are already sufficient for API testing; do not install more tools than you need.

Important version check. The official Vue quick-start currently states Node.js `^22.18.0 || >=24.12.0`. Choose a current Node 24 LTS installer that meets it. An older Node 22 installation may run some Vite projects but not the latest Vue scaffolding; resolve version warnings while still online.

Do this now in Windows PowerShell

```
# Open VS Code terminal in the folder extracted from this kit.
node -v
```

```
npm -v
git --version

# IMPORTANT: do this while internet still works.
npm install

# This produces node_modules/ and package-lock.json.
Copy-Item .env.example .env
npm test
npm run build

# Prepare two terminals before a dry run.
# Terminal A: npm run api
# Terminal B: npm run dev
```

Back up the entire installed folder, including `node_modules` and `package-lock.json`, to your laptop and an external drive. If copying to a different operating system or CPU architecture, native dependencies may need to be reinstalled. A source ZIP without dependencies is **not** a guarantee that Vue works offline. The npm cache is useful as a secondary backup but might not contain every transitive package. `npm ci --offline` works only when the lockfile and all necessary package tarballs already exist in the cache. Your strongest test is **switch off Wi-Fi and rerun the app and build on the assessment laptop**.

No install possible? Open `offline-fallback/index.html` directly in a browser. This shows HTML, CSS, JavaScript, DOM methods, localStorage CRUD, and responsive layout. It does **not** demonstrate Vue or live API competency.

2. Technology map: what each thing does

Technology	What it is	Actual role in this project
HTML	Markup language	Headings, forms, buttons, labels, cards, semantic structure
CSS	Stylesheet language	Grid, spacing, colors, responsive breakpoints, focus styles
JavaScript	Programming language	Logic, events, validation, async requests, data conversion
Vue 3	JavaScript UI framework	Connects data/state to HTML templates automatically
Vite	Development server and build tool	Fast local development, bundles Vue app into `dist/`
Node.js	JavaScript runtime outside the browser	Runs Vite and optional local practice API
npm	Package manager shipped with Node	Installs dependencies and runs project scripts
Git	Local version control	Records changes and lets you inspect/revert versions
HTTP / Fetch / JSON	Web communication and data format	Exchanges product information with a backend API
Backend/database	Server-side components	Store and validate authoritative records; **not** part of Vue

Memory aid: HTML = structure. CSS = appearance. JavaScript = behavior. Vue = a framework to organize JavaScript, HTML-like templates, and state. Vite = tool that builds and serves Vue. Git = history. API = contract between the frontend and backend.

3. Project creation: from a blank folder (online only)

If the assessment provides a blank machine and internet is permitted, the official Vue starter is:

```
npm create vue@latest
# Choose a project name; JavaScript (not TypeScript) is sufficient here.
# Optional Router / Pinia / test tooling only if the task needs them.
cd YOUR_PROJECT_NAME
npm install
npm run dev
```

A straightforward alternative is the Vite Vue template:

```
npm create vite@latest online-palengke -- --template vue
cd online-palengke
npm install
npm run dev
```

For this ZIP, DO NOT scaffold over it. Extract the ZIP; open the `Palengke_NCIII_Offline_Kit` folder; run `npm install` while online; copy `.env.example` to `.env`; then follow the startup commands. The provided package versions are compatible target ranges; `npm install` resolves them and generates your lockfile.

4. Folder-by-folder: exactly where each snippet goes

```
Palengke_NCIII_Offline_Kit/
  index.html                  <-- browser entry HTML
  package.json                <-- npm dependencies and scripts
  vite.config.js              <-- Vite + Vue plugin
  .env.example                <-- example server URL
  .env                        <-- create locally; not committed
  src/
    main.js                   <-- mount Vue and import CSS
    App.vue                   <-- Vue HTML template + JavaScript logic
    style.css                 <-- responsive site design
    services/
      productService.js       <-- live API HTTP requests
      demoStore.js            <-- explicit offline localStorage mode
    utils/
      validation.js           <-- input-validation functions
  mock-api/server.cjs         <-- optional practice API; no database
  tests/validation.test.mjs   <-- unit tests
  offline-fallback/index.html <-- no-dependency browser-only demo
  docs/                       <-- contract, project plan, test plan
```

Rule: HTML for global document setup goes in `index.html`; Vue content and click handlers go in `src/App.vue`; site-wide styling goes in `src/style.css`; network requests should be gathered in `src/services/productService.js`. Do not write `fetch()` in every card. Do not place browser secrets in `.env` variables prefixed `VITE_`.

Snippet A. Main HTML: paste into `index.html`

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Online Palengke | NC III Practice</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

What it does: `#app` is the element Vue controls. `viewport` allows responsive layout on mobile. `type="module"` loads JavaScript modules. In Vite, `index.html` belongs at the project root, not inside `src`.

Snippet B. Mount Vue: paste into `src/main.js`

```javascript
import { createApp } from 'vue'
import App from './App.vue'
import './style.css'

createApp(App).mount('#app')
```

What it does: imports the Vue framework, imports the main component, loads global CSS, then mounts Vue onto `<div id="app"></div>`. Check the relative paths carefully.

Snippet C. Small Vue component: paste into `src/App.vue` for an isolated first test

```
<script setup>
import { ref } from 'vue'
const productName = ref('Tomatoes')
const price = ref(45)
function increasePrice() {
  price.value += 5
}
</script>

<template>
  <main>
    <h1>{{ productName }}</h1>
    <p class="product-price">₱{{ price }}</p>
    <button type="button" @click="increasePrice">+ ₱5</button>
  </main>
</template>
```

Where: all three blocks belong to one `.vue` component. This is a *learning replacement* for `App.vue`, not an additional snippet to paste under the completed application. Restore the supplied complete `App.vue` from the ZIP to run the full CRUD project.

Why this works: `ref(45)` creates reactive state; `price.value += 5` changes the value in JavaScript; `{{ price }}` updates the page; `@click` handles the button. In Vue templates the ref is auto-unwrapped; in `<script setup>` use `.value`.

Snippet D. CSS: paste into `src/style.css`

```css
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; }
.product-price { color: #17603c; font-size: 1.5rem; font-weight: bold; }
button { padding: .7rem 1rem; border-radius: .6rem; cursor: pointer; }

/* Wide screens: multiple cards. Mobile: one card per row. */
.product-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}
@media (max-width: 700px) {
  .product-grid { grid-template-columns: 1fr; }
}
```

Where: this is a *miniature exercise*, not a replacement for the project's complete `src/style.css`. The companion project contains full responsive styles. A CSS class such as `.product-price` matches `class="product-price"` in HTML; an ID selector would start with `#`.

5. Build the cards using Vue instead of manual DOM updates

Place: inside the `<template>` of `src/App.vue`, replacing the simple exercise template or studying the complete one already provided.

```html
<div class="product-grid">
  <article v-for="product in products" :key="product.id"
           class="product-card" :data-product-id="product.id">
    <h3>{{ product.name }}</h3>
    <p class="product-description">{{ product.description }}</p>
    <p class="product-stock">Stock: {{ product.stock }}</p>
    <p class="product-price">₱{{ product.price }}</p>
    <button type="button" @click="increasePrice(product)">+ ₱5</button>
  </article>
</div>
```

Necessary JavaScript in `<script setup>`:

```javascript
import { ref } from 'vue'
const products = ref([
  { id: 1, name: 'Tomatoes', description: 'Fresh', price: 45, stock: 10 },
  { id: 2, name: 'Pechay', description: 'Vegetable', price: 25, stock: 8 },
])
function increasePrice(product) {
  product.price += 5 // For purely local exercise only.
}
```

Precisely why the pieces are connected: `v-for` repeats a card for each object in `products`; `product.id` is its identity; `:key="product.id"` helps Vue track list entries; `:data-product-id="product.id"` writes an optional HTML data attribute useful for inspection; `.product-price` is a CSS class; `{{ product.price }}` shows reactive data. You do not need `card.querySelector('.product-price')` to update a Vue card. With API integration, **replace local mutation with a server PATCH and re-fetch**.

Your earlier DOM method, contrasted:

```javascript
// Plain JavaScript DOM approach (NOT needed in normal Vue state updates):
const card = document.querySelector('[data-product-id="1"]')
const priceElement = card?.querySelector('.product-price')
if (priceElement) priceElement.textContent = '₱50'
```

That updates only the displayed text; it does not update backend data or necessarily Vue's state. In the full app, successful API mutation is followed by `loadProducts()`, which updates the Vue `products` ref with the backend's actual records.

6. API integration: where HTTP belongs

First read the API specification from the assessor. This practice contract uses `GET /products`, `POST /products`, `PATCH /products/:id`, and `DELETE /products/:id`. Do not assume those paths or JSON keys on a different backend. This is an example, not an assessment-center guarantee.

Place server address: copy `.env.example` to `.env` at the project root:

```
VITE_API_BASE_URL=http://127.0.0.1:3000
```

Never use `https://127.0.0.1:3000` with the supplied HTTP-only mock API. That produced the kind of `ERR_SSL_PROTOCOL_ERROR` you encountered before. `localhost` means the current computer, not the public internet.

Place this standalone GET practice snippet: `src/services/productService.js` (the complete source already provides improved timeout/status handling).

```
const API_BASE = import.meta.env.VITE_API_BASE_URL
export async function getProducts() {
  const response = await fetch(`${API_BASE}/products`)
  if (!response.ok) throw new Error(`GET failed: HTTP ${response.status}`)
  return await response.json()
}
```

Place the following caller in `<script setup>` in `src/App.vue`:

```
import { onMounted, ref } from 'vue'
import { getProducts } from './services/productService'
const products = ref([])
const errorMessage = ref('')
async function loadProducts() {
  try {
    errorMessage.value = ''
    products.value = await getProducts()
  } catch (error) {
    errorMessage.value = error.message
  }
}
onMounted(loadProducts)
```

This is a learning excerpt. **Do not duplicate** declarations if using the complete app, which already has `loadProducts`, `onMounted`, and the shared service object.

REST quick reference

Operation	Endpoint	Data sent	Success may look like
Read all	`GET /products`	No request body	`200` and array of products
Create	`POST /products`	New product JSON	`201` and created product
Edit	`PATCH /products/1`	`{ "price": 50 }`	`200` and updated product
Delete	`DELETE /products/1`	No body	`204` with no body

Keep method/path/body aligned. `PATCH /product/1` and `PATCH /products/1` are different URLs. A body such as `{ price: 50 }` must be JSON encoded. Inspect the *Network* tab for the request payload, response status, and server response. Do not invent an update only in the UI when the request fails.

Snippet E. Reusable Fetch wrapper (put in `src/services/productService.js`)

```
async function apiRequest(path, options = {}) {
  const response = await fetch(`http://127.0.0.1:3000${path}`, {
    ...options,
    headers: { 'Content-Type': 'application/json', ...options.headers },
  })
  const text = await response.text()
  const data = text ? JSON.parse(text) : null // Handles 204.
  if (!response.ok) throw new Error(data?.message || `HTTP ${response.status}`)
  return data
}

// Illustrative use:
await apiRequest('/products/1', {
  method: 'PATCH',
  body: JSON.stringify({ price: 50 }),
})
```

The actual project wrapper also times out requests, handles invalid JSON, and reports network failure. Calling `response.json()` without checking for an empty body can crash a perfectly successful `204 No Content` deletion.

Snippet F. Reliable price increment (put in `<script setup>` of `src/App.vue`)

```
async function increasePrice(product) {
  const nextPrice = Number(product.price) + 5
  if (!Number.isFinite(nextPrice) || nextPrice < 0) return
  try {
    await productService.update(product.id, { price: nextPrice })
    await loadProducts() // Uses server result, not a frontend guess.
  } catch (error) {
    errorMessage.value = `Unable to increase price: ${error.message}`
  }
}
```

The actual `App.vue` version also disables duplicate requests, shows success messages only after a successful reload, and maintains a clear API-versus-demo switch. **Important security note:** frontend validation does not protect the API from a direct crafted request. A real backend must repeat validation and enforce authorization.

Snippet G. Reliable delete (put in `<script setup>` of `src/App.vue`)

```
async function deleteProduct(product) {
  if (!window.confirm(`Delete ${product.name}?`)) return
  try {
    await productService.remove(product.id)
    await loadProducts() // Server confirms what still exists.
  } catch (error) {
    errorMessage.value = `Unable to delete: ${error.message}`
  }
}
```

Button in the card: `<button type="button" @click="deleteProduct(product)">Delete</button>`. If your API is down, an error should appear and no fake deletion should be shown as successful. If a backend permanently deletes the record but the following GET fails, the app should report the reload failure instead of pretending verification was completed.

7. CRUD forms and input validation

Place a form inside `<template>` of `src/App.vue`. This simplified code is for understanding; the full example supports Add and Edit using one form.

```
<form @submit.prevent="saveProduct">
  <label>Product name <input v-model.trim="form.name" required /></label>
  <label>Price <input v-model.number="form.price" type="number" min="0" /></label>
  <label>Stock <input v-model.number="form.stock" type="number" min="0" step="1" /></label>
  <button type="submit">Save</button>
</form>
```

Place state in `<script setup>` of `src/App.vue`:

```
import { reactive } from 'vue'
const form = reactive({ name: '', price: 0, stock: 0, description: '' })
```

Place reusable input checking in `src/utils/validation.js`:

```
export function validateProduct(product) {
  if (String(product.name).trim().length < 2) return 'Name too short.'
  if (!Number.isFinite(Number(product.price)) || Number(product.price) < 0)
    return 'Price must not be negative.'
  if (!Number.isSafeInteger(Number(product.stock)) || Number(product.stock) < 0)
    return 'Stock must be a non-negative whole number.'
  return ''
}
```

For user-entered numbers, remember browser inputs produce values that often need converting to numbers; `v-model.number` helps. Check that `stock` is an integer and `price` is finite; `Number.isFinite()` rejects `Infinity` and `NaN`. In an actual server-backed app, enforce these constraints on the server too. This kit's local mock API demonstrates that extra validation.

8. Run the supplied local API without internet

Node's built-in `http` module runs the example API without Express, a database, or an extra backend dependency.

```
# From the extracted project root:
npm run api
# In a SECOND terminal:
npm run dev
# Open the browser at:
# http://127.0.0.1:5173
```

The mock API listens at `http://127.0.0.1:3000`. To inspect records in the browser, open `http://127.0.0.1:3000/products`. `GET /health` can show whether the process is running. The server's records are held **in memory and reset when it restarts**. That is okay for HTTP practice but does not demonstrate database integration.

How to verify a real PATCH: open DevTools (`F12`) -> *Network* -> click `+ ₱5` -> select request `/products/1` -> verify Method `PATCH`, Request Payload `{ "price": 50 }`, successful status, then subsequent `GET /products`. The displayed new price comes from reloaded backend data.

API unavailable versus public internet unavailable

- No internet connection, but the local Node mock API is running: **GET/POST/PATCH/DELETE still work** because both processes run on your laptop.
- API process stopped or on wrong port: the app cannot retrieve new server records. It shows a visible error; it does not silently save to localStorage.
- Switch explicitly to **Offline demo**: CRUD runs against `src/services/demoStore.js` (browser localStorage). This proves only Vue UI behavior, not the API.
- Use the no-install fallback: pure HTML, CSS, DOM, and JS functions still work with the file opened directly.

9. JavaScript and Vue essentials you should explain aloud

Syntax	Meaning	Example
`const`	Variable binding that cannot be reassigned	`const price = 10`
`let`	Variable binding that can be reassigned	`let count = 0`
`+=`	Add then assign	`price += 5` means `price = price + 5`
`===`	Strict equality	`id === 1`
`?.`	Optional chaining	`card?.querySelector(...)`
`??`	Default only when null/undefined	`description ?? ''`
`async` / `await`	Work with asynchronous promises	`await fetch(url)`
`try/catch/finally`	Handle failures and cleanup	catch network errors
`ref([])`	Reactive Vue data	`products.value = data`
`computed()`	Derived Vue state	filtered product list
`v-for` and `:key`	Repeat and track rendered nodes	product cards
`v-model`	Two-way form input binding	editing price
`@click`	Vue event handler	call `deleteProduct(product)`
`@submit.prevent`	Submit without page reload	product form
`onMounted()`	Function called after Vue mounts	initial API fetch
`.filter()`	Select matching items	search
`.find()`	Get first matching element	product by ID
`.map()`	Transform collection	names from product objects

JavaScript vs Vue DOM: plain JavaScript uses `document.querySelector()`, `textContent`, `createElement()`, and event listeners; Vue prefers its declarative `<template>` with state and bindings. Both are valid skills; don't mix manual DOM mutation into Vue unnecessarily.

10. Responsive design, accessibility, and browser inspection

Follow the supplied design/mockup first. The practice app has a three-column desktop card grid, two columns on tablets, and one column on narrower phones. The critical responsive techniques are `display: grid`, `minmax(0, 1fr)`, flexible widths, `max-width`, and `@media` breakpoints. Check at approximately 375px, 768px and 1440px, plus one device in landscape. The number is a test viewport, not a guaranteed TESDA-mandated breakpoint.

Accessible HTML basics: use proper headings, descriptive page title, a label for each input, `<button>` rather than clickable `<div>`, keyboard-accessible actions, visible focus, an error with `role="alert"`, and sufficient contrast. Render product text with `{{ }}` rather than `v-html` when showing untrusted strings to avoid introducing HTML injection.

DevTools workflow: *Elements* for HTML/CSS and box model; *Console* for uncaught JavaScript errors; *Network* for API URLs/status/body/CORS; *Device Toolbar* for responsive behavior; *Sources* for breakpoints. `localhost` requests continue to work when the public internet is unavailable, provided the services are local.

11. Git: record your work, even while offline

```
# From the project root:
git init
git status
git add .
git commit -m "Initial Vue palengke frontend"

git switch -c feature-product-crud
# Make and test your code changes.
git add src/App.vue src/services src/utils
git commit -m "Add validated product CRUD and API integration"
git log --oneline -5
git diff
```

For your existing `feature-vue` branch, check `git status` before changing anything. If your working tree is clean, you can safely make a new logical change. **Committing works offline**. Pushing to GitHub or pulling remote updates requires connectivity. Commit evidence and local backups are different: Git alone is not a substitute for an external backup.

Do not commit: `.env` if it contains private configuration, API keys, credentials, `node_modules`, `dist`, or user/private data. `.gitignore` excludes generated directories. Secrets included in `VITE_` environment values are visible in frontend bundles and must never be treated as confidential.

12. Debugging: symptoms -> root causes -> actions

What you see	Most likely cause	First action
`ERR_SSL_PROTOCOL_ERROR`	HTTPS URL against HTTP server	Change `https://127.0.0.1:3000` to `http://127.0.0.1:3000`
`Failed to fetch`	Server down, wrong host/port, CORS, network block	Check `npm run api`, URL, Network/Console
`404 Not Found`	Wrong API path, method, or ID	Compare exact route `/products/:id`
`405 Method Not Allowed`	Backend route exists but method wrong	Check `PATCH` versus `PUT/POST`
`400 Bad Request`	JSON fields/types rejected	Inspect request body, backend validation
`401 Unauthorized`	Missing/expired login credentials	Follow supplied API auth procedure
`403 Forbidden`	Account lacks permission	Check backend authorization; frontend cannot override
`500 Server Error`	Backend exception	Inspect API logs; do not conceal error
`Cannot find module vite`	Packages not installed	`npm install` while online or restore `node_modules`
UI changes but refresh reverts it	Only DOM changed, not persistent data	Use server request and re-fetch
Delete shows stale card	State not reloaded/updated	Refresh from GET after successful DELETE
Template does not update	Data not reactive or wrong binding	Inspect `ref`, `.value`, and `{{ }}`
CORS message	Backend origin permission missing	Fix server CORS policy; browser front end cannot grant it
Port 5173 switches to 5174	Another process already uses port	Stop duplicate server; align mock API CORS origin

Fast troubleshooting sequence: (1) verify the task's API contract; (2) check `node -v`; (3) check both terminal processes; (4) open Network and Console; (5) inspect Request URL, Method, Payload, Status, Response; (6) reproduce in a REST client or PowerShell; (7) fix; (8) re-test and document.

PowerShell no-install API tests

```
Invoke-RestMethod -Uri 'http://127.0.0.1:3000/products' -Method Get

$body = @{ price = 50 } | ConvertTo-Json
Invoke-RestMethod -Uri 'http://127.0.0.1:3000/products/1' `
  -Method Patch -ContentType 'application/json' -Body $body

Invoke-RestMethod -Uri 'http://127.0.0.1:3000/products/1' -Method Delete
```

Use the delete command only on disposable mock/test data. Windows PowerShell versions differ in how they display `204 No Content`, but the server operation can still succeed.

13. Testing protocol (collect actual evidence)

Record each case as **Pass / Fail / Not Run**, expected versus actual result, date, screenshot, and corrective action. The companion `docs/TEST-PLAN.md` has 15 ready-to-check test cases. The minimum sequence worth memorizing is:

1. Start the frontend **and** API. Verify initial GET loads three mock products.
2. Click + ₱5; inspect PATCH body and subsequent GET; confirm price increases.
3. Add, edit, and delete a product; reload and verify changed API data.
4. Enter invalid values (negative price, negative or decimal stock); show validation error.
5. Stop API; click Refresh; verify a user-friendly error and no false success.
6. Switch to Offline demo; verify it is visibly labeled as browser-only storage.
7. Resize to phone/tablet/desktop; tab through buttons and input labels.
8. Run `npm test`, `npm run build`, inspect `dist/`, review Git log.

A browser-only demo is not evidence of backend access. A local mock API is evidence of making HTTP requests but not evidence of integrating an assessor's particular backend. A build command tests compilation, not necessarily every user journey. Unit tests and manual browser tests complement each other.

14. Secure coding and limits of frontend enforcement

Client-side: validate inputs to improve UX; use Vue's safe text interpolation; avoid `v-html` for untrusted content; constrain form types/lengths; handle network errors without leaking secrets; avoid hardcoded credentials; use HTTPS for real deployed services; prevent duplicate submissions.

Backend responsibility: validate all API requests *again*; implement real authentication and authorization; enforce ownership and roles; use parameterized queries for databases; set CORS and session policy; rate-limit or log appropriate operations; protect tokens and secrets. A user can bypass any frontend check with a direct API request. Vue cannot enforce database security by itself.

If the provided API uses authentication: first read its documentation: maybe a cookie session, maybe `Authorization: Bearer <token>`, maybe another mechanism. An authorized API call might look like:

```
const response = await fetch('/api/products', {
  headers: { Authorization: `Bearer ${accessToken}` },
})
```

This is only a generic example. Never invent credentials or store long-term secrets in source code; prefer the documented authentication flow and server-side access controls. The supplied local mock API does not implement login/authorization and is designed only for loopback practice.

15. Performance and submission optimization

Use efficient responsive CSS; avoid repeated network requests while a request is already pending; render lists with stable `:key`; compress and resize images; avoid unnecessary third-party libraries/CDNs; check the browser console; test on a phone-size viewport. `npm run build` generates optimized static assets under `dist/`. A CDN can improve deployment speed, but **an offline assessment should not depend on a remote CDN**: package local assets/dependencies instead. Use only images/assets you may legitimately use.

16. Simple documentation you can present to an assessor

For the supplied Palengke example, include these deliverables **subject to the actual instructions**:

Evidence	Where it is in the kit	What it proves
Project requirements and schedule	`docs/PROJECT-PLAN.md`	Plan and iterative methodology
Architecture / data flow	Project plan + this handbook	Vue UI, service, backend, local demo boundaries
Installation instructions	`README.md`	Setup/reproducibility
API endpoint specification	`docs/API-CONTRACT.md`	HTTP contract awareness
HTML/CSS/Vue/JS working source	`index.html`, `src/`	Responsive interface and interactive behavior
Manual test cases	`docs/TEST-PLAN.md`	Testing strategy and recorded results
Unit tests	`tests/validation.test.mjs`	Validations verified independently
Version control	Local Git commits	Work history
Final build	`dist/` after `npm run build`	Compilable production bundle

Evidence	Where it is in the kit	What it proves
Screenshot evidence	Capture on your laptop	UI at mobile/desktop, API Network requests

30-second explanation you can deliver

> "I built the frontend using HTML, CSS, JavaScript, and Vue 3. Vite serves and builds the application. Product cards are generated from reactive Vue state. `productService.js` sends the HTTP requests to the API. After modifying backend data, the app reloads products to use the server as the source of truth. I added frontend validation, network error handling, responsive layouts, Git commits, and tests. The offline demo is separate from API mode."

17. Timed hands-on drills

Drill A — HTML/CSS (20 minutes): build a header, three product cards, a form, and a footer. Add a one-column mobile breakpoint and accessible labels. Inspect 375px width. **Drill B — JavaScript (20 minutes):** use `const`, arrays, `filter`, events, form validation, and a + ₱5 button in pure DOM or Vue state. **Drill C — Vue (30 minutes):** make `ref([])`, `onMounted()`, `v-for`, `:key`, `v-model`, and an edit form. **Drill D — API (35 minutes):** connect GET/PATCH/POST/DELETE and verify Network; stop the API and demonstrate error handling. **Drill E — evidence (20 minutes):** record a Git commit, run tests and build, write a brief test report.

A completed project is useful, but real readiness means you can **reconstruct the important parts without copying blindly**, find endpoint mistakes, and explain each action in plain language.

18. Assessment-day checklist

- ☐ The computer boots, battery/power plan is ready, Node/npm, VS Code, Git, browser work.
- ☐ Offline test passed for `npm run dev`, `npm run api`, `npm test`, `npm run build`.
- ☐ `node_modules`, lockfile, source ZIP, docs, and installers are backed up.
- ☐ You can start from a blank Vue component and create a card with `v-for`.
- ☐ You know where to put HTML, CSS, JavaScript, Vue logic, and API service code.
- ☐ You know why `http://` and `https://` cannot be interchanged.
- ☐ You can diagnose 400/401/403/404/500/network errors from DevTools.
- ☐ You can validate numbers, handle empty or 204 responses, and avoid false success.
- ☐ You can test responsive layouts and keyboard usage.
- ☐ You can show Git history, project plan, API contract, test evidence, and build output.
- ☐ You will follow the assessor's given design, instructions, API specifications, and allowed-tool rules first.

19. Official reference URLs (download as HTML/PDF while still online)

- Vue Quick Start: https://vuejs.org/guide/quick-start.html
- Vue IDE Tooling: https://vuejs.org/guide/scaling-up/tooling
- Vite Getting Started: https://vite.dev/guide/
- Node.js installers: https://nodejs.org/en/download
- VS Code extensions and VSIX: https://code.visualstudio.com/docs/configure/extensions/extension-marketplace
- npm offline setting reference: https://docs.npmjs.com/cli/using-npm/config/
- Git documentation: https://git-scm.com/doc
- Browser DevTools guide: https://developer.chrome.com/docs/devtools/

Assessment source documents supplied with your project: *2. SAG - Web Development (Front-End) NC III.pdf* (Self-Assessment Guide, pp. 1-5) and *TR-Web Development (Front End) NC III.pdf* (Training Regulations; core units ICT251307 and ICT251308; see their Core Competencies sections). The code, exercises, schedules, examples, and simulated server in this handbook are independent instructional additions.

Appendix A. Essential complete project components

The companion source ZIP is the **authoritative full code listing**. To prevent transcription mistakes, open the relevant file instead of typing the long Vue template or entire CSS stylesheet from the handbook. The ZIP contains the working source structure and the documentation in editable Markdown. The four most important source files to study in order are:

1. `src/App.vue` — reactive state, HTML template, CRUD handlers, and errors.
2. `src/services/productService.js` — all fetch methods, timeout, response status and JSON handling.
3. `src/utils/validation.js` — input validation and payload types.

4. `src/style.css` — layout and responsive media queries.

Mini tracing exercise: click `+ ₱5` on a card -> `@click="increasePrice(product)"` -> `productService.update(product.id, {price: nextPrice})` -> Fetch `PATCH /products/<id>` -> server changes record -> `loadProducts()` performs GET -> `products.value` updates -> Vue re-renders `{{ money(product.price) }}`. Practice tracing this until you can explain it without notes.

Appendix B. Glossary and troubleshooting keywords

API: interface contract for sending/receiving data. **Endpoint:** URL for an operation. **Request payload:** JSON body sent to server. **Response:** status code, headers, body. **CORS:** browser-enforced cross-origin permission configured by server. **DOM:** browser's live document object model. **Reactive state:** data changes that trigger Vue updates. **CRUD:** create/read/update/delete. **LocalStorage:** persistent browser-specific key/value storage; not a database backend. **Build:** compilation/bundling into deployable static assets. **Commit:** local saved Git history item. **Mock API:** temporary simulation of a backend to practice HTTP flows. **Server source of truth:** show backend-confirmed data, not guessed client changes.

The practical standard: assess the actual assigned requirements, run what you build, demonstrate each relevant behavior, report the limitations, and keep evidence of test results.